OPEN ROLE · FULL STACK ENGINEER

Ego,Centric Video Capture System Technical Assignment

Build a React Native Android app that captures first,person video, persists it reliably, and syncs it to AWS — mirroring the actual infrastructure powering Locara's data pipeline.

EVALUATION PILLARS

25%	25%	25%
APK Engineering	**Database Design**	**AWS Infrastructure**
Stable recording, offline resilience, upload queue correctness	Schema clarity, index rationale, migration strategy, query efficiency	Security posture, presigned URL flow, lifecycle cost strategy

> Candidates who attempt all three pillars ,even partially ,rank above those who only complete the app UI. A well, designed DB schema with clear justification outweighs a feature, complete app with no schema thought.

01 Authentication

REQUIREMENTS

- Phone number or email,based login — mock auth is acceptable
- Session persistence across app restarts
- Secure local token storage (AsyncStorage with encryption or equivalent)

02 Video Capture Module

FUNCTIONAL REQUIREMENTS

- Start / stop recording with real,time timer indicator
- Front / rear camera toggle
- Configurable max duration (default: 60 seconds)
- Auto,save on recording completion or duration limit

MANDATORY METADATA — CAPTURED PER RECORDING

FIELD	TYPE	NOTES
video_id	**UUID v4**	Generated on capture start
worker_id	**String**	From authenticated session
started_at	**ISO 8601**	Timestamp at record,press
ended_at	**ISO 8601**	Timestamp at stop or limit
duration_ms	**Integer**	Millisecond precision
file_size_bytes	**Integer**	Post,recording file size
fps	**Float**	From camera API or derived
fps_tier	**Enum**	low <20 / standard 20,30 / high >30
device_model	**String**	e.g. Samsung Galaxy A52
os_version	**String**	Android version string
resolution	**String**	e.g. 1920x1080
local_path	**String**	Absolute path on device

OPTIONAL METADATA (BONUS SIGNAL)

- GPS coordinates (lat/lng) at recording start
- Battery level at recording start and end
- Network type at upload time: wifi, cellular, none

03 Local Database Layer · Weighted 25%

Use SQLite (via react,native,quick,sqlite or expo,sqlite) or Realm. Your schema must be designed with production scale in mind from the start.

SCHEMA DELIVERABLE

- Primary keys and foreign keys explicitly defined
- Indexes with written justification — which columns and why
- A metadata JSON / TEXT column for extensible fields
- An upload_state column: pending, uploading, uploaded, failed
- Upload retry tracking: attempt_count, last_error, last_attempted_at

MIGRATION STRATEGY

Document in your README how you would handle schema migrations in production — for example, adding a gps_accuracy column to 50K existing rows on an app update. Code is optional; a clear written plan is required.

QUERY EFFICIENCY

Include at least one example of a query you optimised and why — e.g. pagination for the video list, or filtering by upload status across a large table.

04 Upload & Sync Engine · Weighted 25%

UPLOAD FLOW

App → Backend API → Presigned URL → App PUT to S3

You may mock the backend API — a simple local Node.js or Python function that generates a presigned URL is sufficient.

UPLOAD STATES

● PENDING	● UPLOADING	● UPLOADED	● FAILED
Recorded, not yet attempted	Active transfer in progress	Confirmed by S3 or backend	Max retries — manual retry available

RELIABILITY REQUIREMENTS

- Queue state persists across app kill and restart
- Automatic retry with exponential backoff — 2s → 4s → 8s → max 64s
- No duplicate S3 uploads — use video_id as idempotency key
- Interrupted uploads retry from the start (multipart is a bonus)
- A video that reaches UPLOADED must never revert to PENDING

05 AWS Infrastructure Design · Weighted 25%

You are not required to provision live AWS infrastructure. Document your design clearly in your README or a separate INFRA.md. A working Terraform or CDK snippet is a strong signal of depth.

Q1 — S3 Bucket Design
How would you structure the S3 key namespace for 10K workers uploading ~20 videos/day? Provide an example key format with justification. Would you use one bucket or multiple? Why?

Q2 — IAM & Security
What IAM policy would the presigned URL generator use? Provide a minimal least,privilege policy. How would you scope URLs so Worker A cannot overwrite Worker B's data? What's an appropriate TTL for a 60,second video upload?

Q3 — Storage Cost Strategy
At 10K workers x 20 videos/day x avg 50MB — estimate monthly S3 cost after 90 days. What lifecycle policy would you apply? Would you use S3 Intelligent,Tiering? Justify your choice.

Q4 — Upload Confirmation
How does the app confirm the upload actually succeeded on S3's side? Describe your approach: S3 event → Lambda → DB update, presigned URL + ETag verification, or something else. Justify your choice.

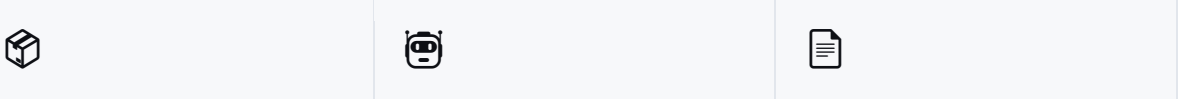
Q5 ★ — Bonus: Presigned URL Generator
Provide a working presigned_url_generator.py or equivalent that generates a scoped PUT URL for a given video_id and worker_id.

06 Video Management Dashboard

A functional screen — UI polish is not evaluated here.

- Paginated list of all recorded videos
- Per,video: duration, file size, FPS tier, upload status
- Actions: retry failed upload, delete local file
- Clear visual differentiation of upload states

07 Deliverables



GitHub Repository

→ Clean folder structure: services/, db/, screens/
→ TypeScript throughout (preferred)
→ .env.example with all env vars
→ README with setup, architecture, DB & AWS rationale

Android APK

→ Installable debug build
→ Tested on Android 10+ (API 29+)
→ Note which device / emulator was used

Technical Doc (2–4 pages)

→ System architecture diagram
→ Data flow: capture → DB → queue → S3
→ DB schema with index justification
→ AWS design decisions
→ Retry and idempotency strategy
→ Scalability: what breaks first at 10K users

08 Evaluation Rubric

DIMENSION	WEIGHT	WHAT WE LOOK FOR
APK Engineering	25%	Stable recording, offline resilience, upload queue correctness
Database Design	25%	Schema clarity, indexing rationale, migration strategy
AWS Infrastructure	25%	Security posture, cost awareness, presigned URL flow
Code Quality	15%	Modularity, error handling, TypeScript correctness
Documentation	10%	Clarity of architecture and trade,off reasoning

This assignment mirrors a real system Locara Labs operates. Strong candidates often ask follow,up questions about the real pipeline — that's actively encouraged. Send your GitHub link + APK with a brief 3–5 sentence note on what you prioritised and what you'd improve given more time.